PYTHON MULTIPROCESSING

Процеси в Python

Паралелізм без обмежень

Справжня багатозадачність, обхід GIL, механізми fork/spawn та ефективна міжпроцесна взаємодія.

Сумець Світлана

2026

Процеси vs Потоки: Коли що обирати

Процеси

CPU-Bound (Обчислення)

Повна ізоляція пам'яті

Ідеально для:

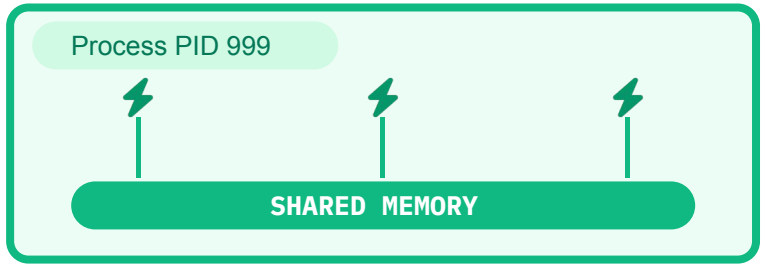
Важких математичних обчислень, обробки відео, ML. Кожен процес має свій GIL, що дозволяє використовувати всі ядра CPU реально паралельно.

- Обхід GIL (Паралелізм)
- Важкий запуск та IPC

VS

Потоки

I/O-Bound (Очікування)



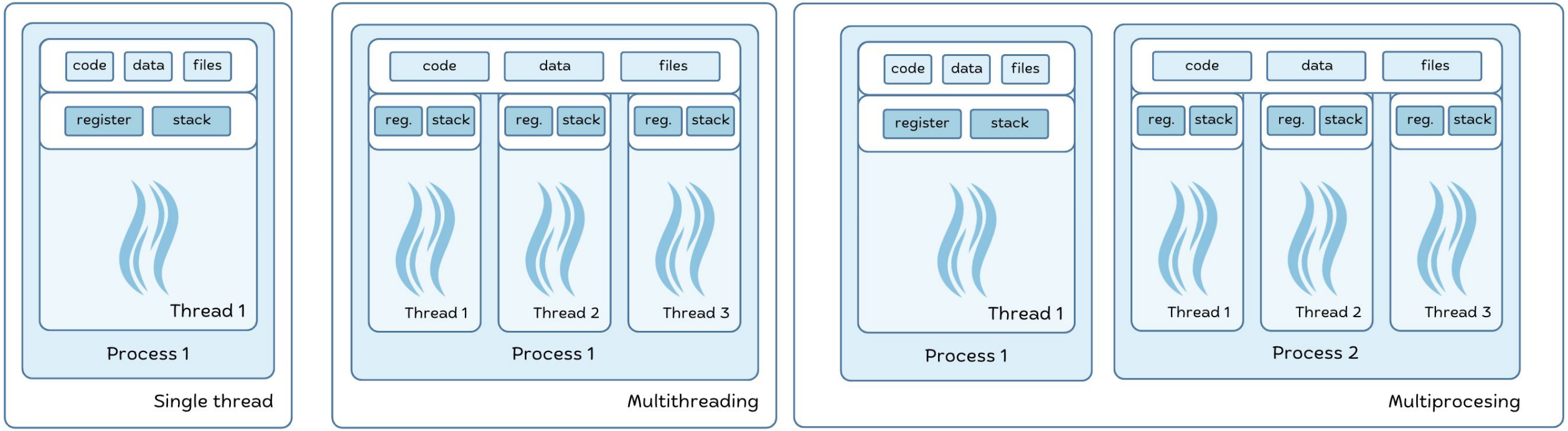
GIL перемикає контекст

Ідеально для:

Мережевих запитів, роботи з файлами, очікування вводу. Потоки ділять пам'ять, що спрощує обмін даними, але вимагає синхронізації.

- Легковісність та швидкий старт
- Обмежені GIL (Конкурентність)

Багатопроцесність vs Багатопоточність



Один процес

GIL обмежує

Багатопоточність (Threading)

Всі потоки живуть в одному адресному просторі. GIL дозволяє виконуватись лише одному потоку байт-коду в конкретний момент часу.

Кілька процесів

Повний паралелізм

Багатопроцесність (Multiprocessing)

Кожен процес має власну копію інтерпретатора, власну пам'ять і власний GIL. Вони не блокують один одного.

Що таке процес?

> Процес - це незалежний екземпляр програми під час виконання. Це ізольована одиниця, яка має повний контроль над своїми ресурсами та пам'яттю.

Власний адресний простір

Пам'ять одного процесу повністю закрита для інших. Змінні та об'єкти не можна "побачити" ззовні без спеціальних механізмів (IPC).

Унікальні атрибути

Кожен процес має свій PID (ідентифікатор), таблицю дескрипторів файлів, змінні оточення та контекст виконання.

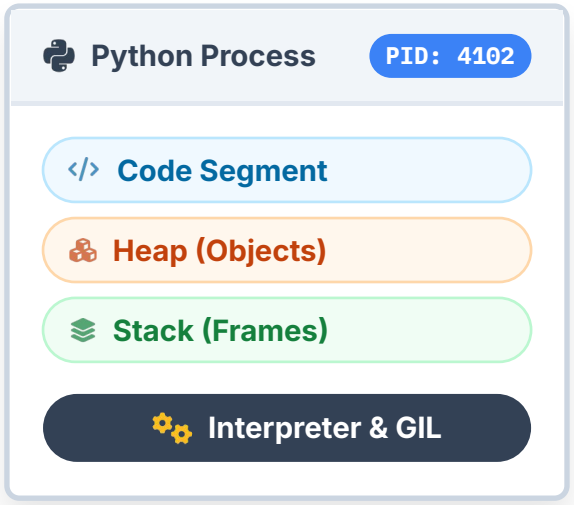
Безпека та Ізоляція

Критична помилка (crash) в одному процесі не "вбиває" всю програму. Це гарантує високу стабільність системи.

Обхід GIL

Кожен процес запускає свій інтерпретатор Python. Це дозволяє використовувати всі ядра CPU на 100% для важких обчислень.

АНАТОМІЯ ПРОЦЕСУ



Пам'ятайте: Процес "важчий" за потік. Створення нового процесу вимагає копіювання або створення нового адресного простору, що займає час.

Механізм fork(): Клонування

Алгоритм роботи fork()

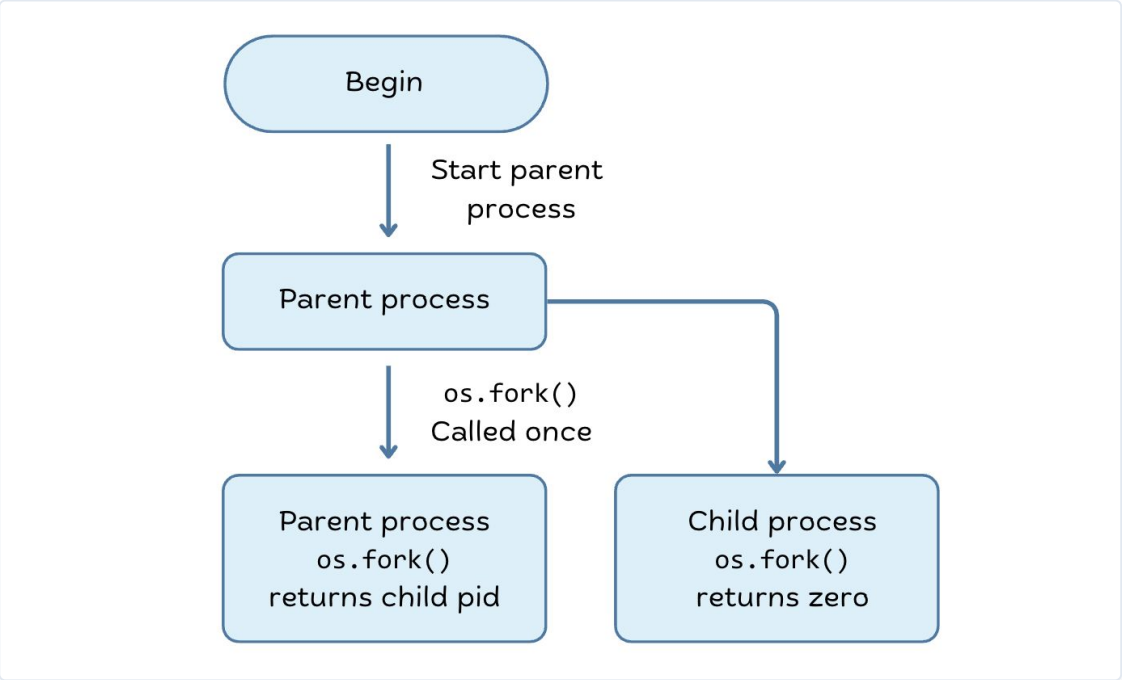
1. **Виклик функції**
   Батьківський процес ініціює системний виклик os.fork(). Виконання призупиняється для дублювання.

2. **Клонування метаданих**
   ОС створює точну копію адресного простору, але віртуально. Фізична пам'ять не копіюється (оптимізація).

3. **Розгалуження (Branching)**
   З'являється новий процес (Child). Обидва процеси знаходяться в одній точці коду, але мають різні PID.

4. **Copy-on-Write (COW)**
   Процеси читають спільну фізичну пам'ять. Реальне копіювання сторінки відбувається лише в момент спроби запису даних.

5. **Повернення значень**
   Батько отримує PID дитини (>0), дитина отримує 0. Це дозволяє розмежувати логіку виконання.



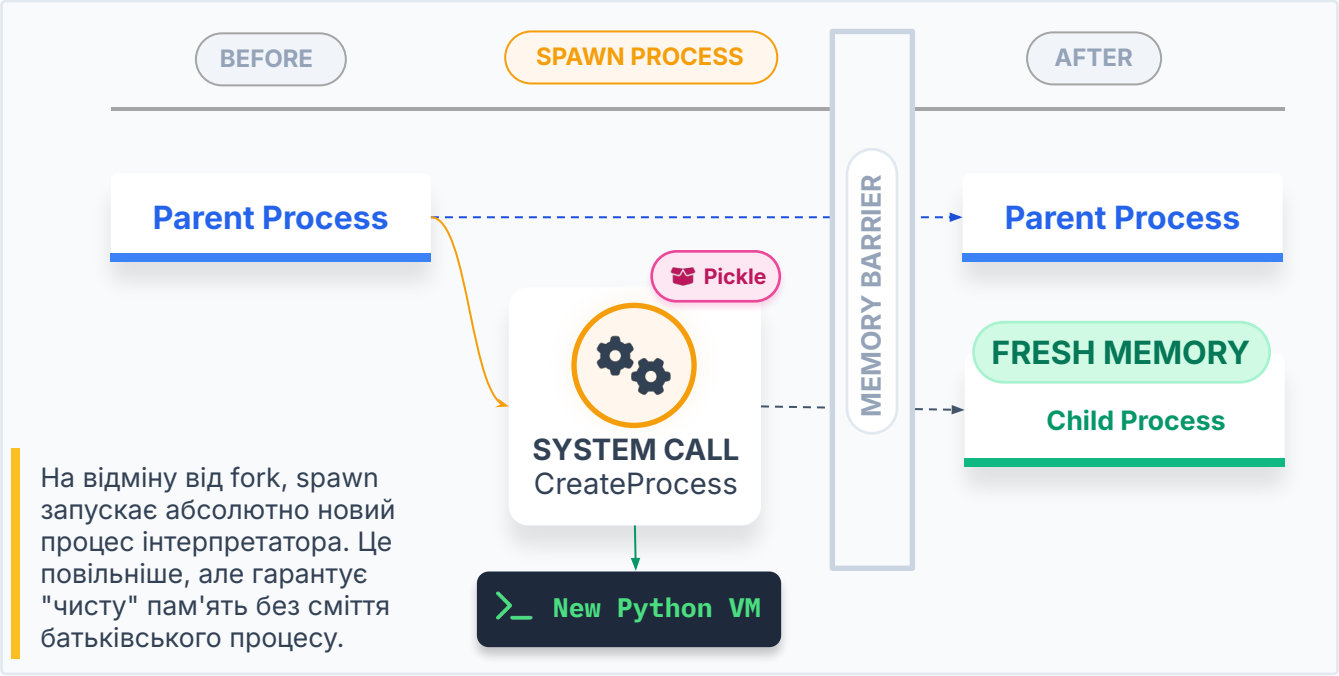
UNIX ONLY

```
import os

pid = os.fork()
if pid > 0:
  print(f"Child is {pid}")
else:
  print("I am new!")
```

Механізм spawn(): Чистий старт

Алгоритм spawn()

1. **Системний виклик**
   Батьківський процес ініціює створення через CreateProcess (Win) або posix_spawn.
2. **Bootstrap (Старт VM)**
   Запускається новий інтерпретатор Python. Це ресурсомістка операція (Slow Start).
3. **Серіалізація (Pickle)**
   Всі аргументи та дані для child-процесу упаковуються через pickle і передаються в pipe.
4. **Імпорт середовища**
   Child-процес імпортує скрипт. В цей момент виконується весь код верхнього рівня.
5. **Повна ізоляція**
   Процеси не мають спільної пам'яті. Всі дані копіюються, а не посилаються.



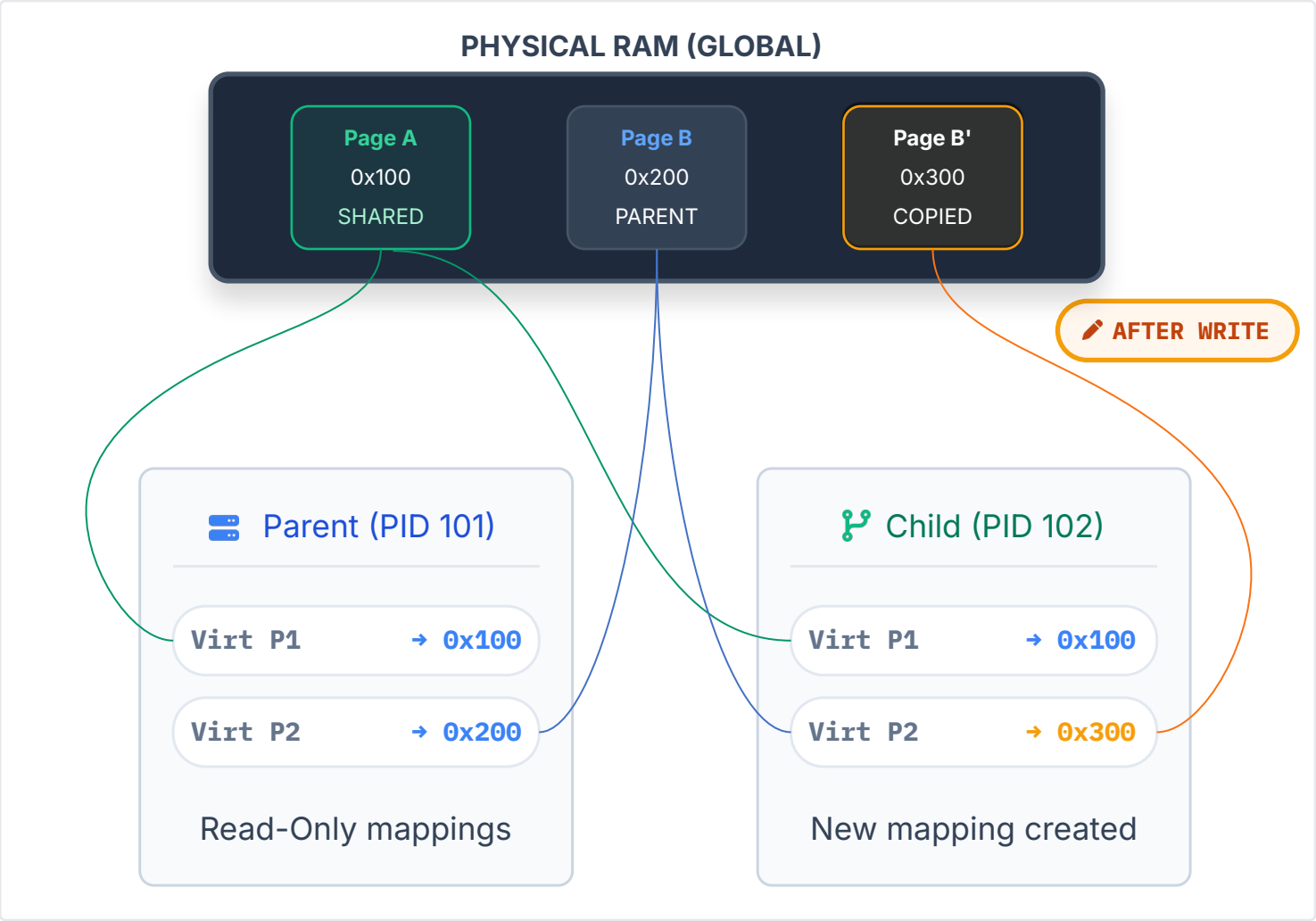

```
import multiprocessing                    SAFE SPAWN PATTERN

def worker():
    print("Work")

if __name__ == '__main__':
    multiprocessing.set_start_method('spawn')
    p = multiprocessing.Process(target=worker)
    p.start()
```

Стандарт для Windows
Єдиний доступний метод на Windows. Також default для macOS з Python 3.8+.

Вимога безпеки
Обов'язкова перевірка if __name__, щоб уникнути рекурсивного створення процесів при імпорті модуля.

Unix Fork	Швидкий
Win Spawn	Безпечний

Copy-on-Write (COW): Економія пам'яті



Як це працює?

При fork() ОС не дублює всю пам'ять миттєво. Батько і дитина використовують спільні фізичні сторінки у режимі "лише для читання".

Коли один з процесів намагається записати дані, ОС перехоплює це, копіює лише цю сторінку в нову область пам'яті і дозволяє запис.

Миттєвий старт
fork() виконується дуже швидко

Економія RAM
Спільні дані не дублюються

linux_fork.py

```
import os
data = [1, 2, 3] # 0x100 (Shared)

pid = os.fork()
if pid == 0: # Child
   print(data) # Reads 0x100
   data[0] = 9 # Triggers Copy -> 0x300
```

Модуль multiprocessing

> Цей модуль є стандартом для створення процесів у Python. Його API навмисно спроектовано схожим на модуль threading для полегшення міграції.

Process & Pool

Process створює один окремий процес, тоді як Pool дозволяє керувати фіксованою групою воркерів для паралельного виконання завдань.

Queue & Pipe

Безпечні канали для обміну даними. Queue — це багатопотокова черга (producer-consumer), а Pipe — швидкий двосторонній канал.

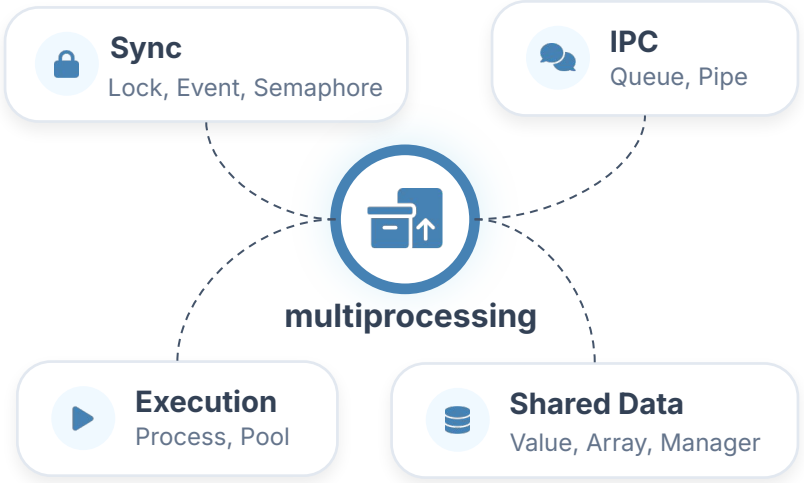
Value & Array

Дозволяють процесам використовувати спільну область пам'яті (Shared Memory) для швидкого доступу до простих типів даних без серіалізації.

Manager

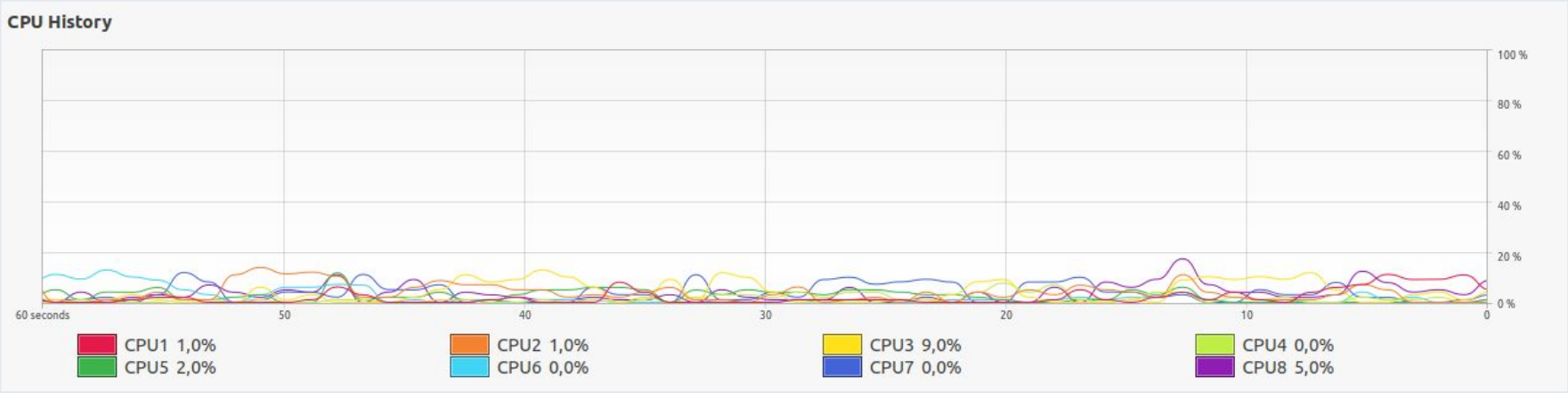
Створює серверний процес, який керує складними об'єктами (списки, словники) і надає доступ до них через проксі-посилання.

СТРУКТУРА МОДУЛЯ



> Порада: Починайте з Pool для простих задач. Якщо потрібен складний контроль стану або обмін даними - використовуйте Manager та Queue.

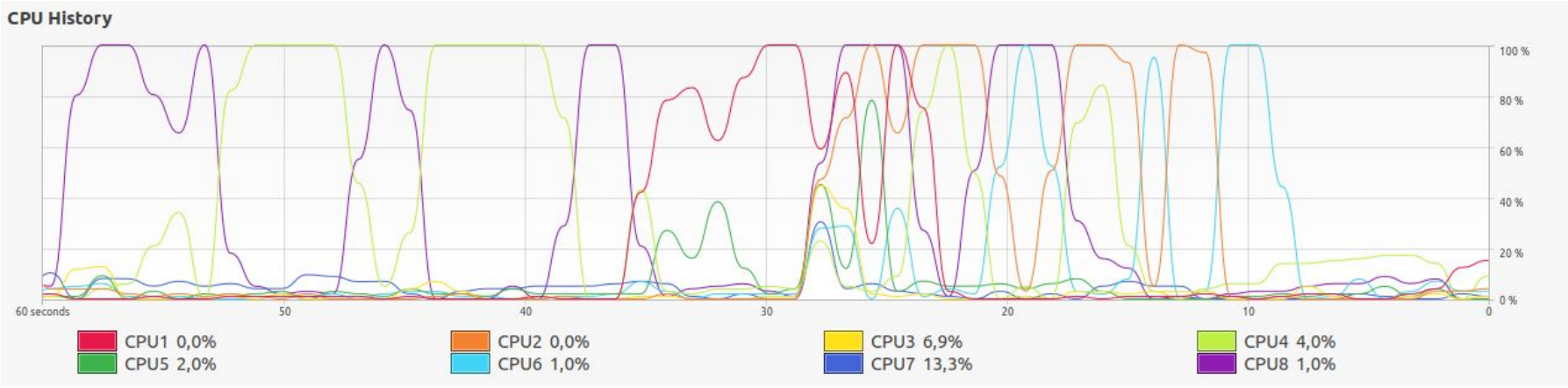
multiprocessing.Pool - With/Without



До запуску

0-2%
Загальне навантаження

Рівномірно
Процеси між ядрами



Послідовне виконання

Паралельне виконання

З процесами

100%
Піки навантаження

Чергування
Планувальник ОС може перерозподіляти ресурси

ProcessPoolExecutor: Високорівневий API

> Сучасний інтерфейс з concurrent.futures, що спрощує запуск процесів та отримання результатів.

Submit & Map

submit() для одиничних задач, map() для паралельної обробки ітерабельних об'єктів (аналог вбудованого map).

Future Objects

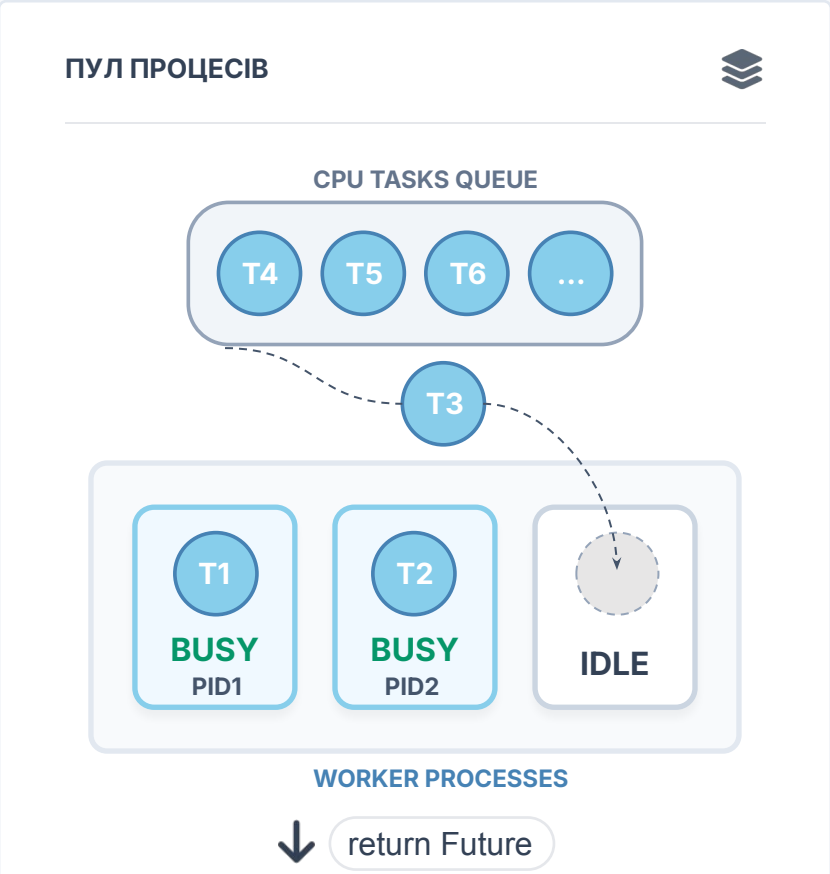
Повертає об'єкти Future, які дозволяють перевіряти статус виконання, отримувати результат або перехоплювати винятки.

Автоматичне управління

Використання контекстного менеджера with гарантує коректне завершення процесів та звільнення ресурсів.

Обробка помилок

Винятки в дочірніх процесах не "ламають" програму, а зберігаються в об'єкті Future до моменту виклику result().



Перевага: ProcessPoolExecutor автоматично серіалізує аргументи та результати, приховуючи складність IPC.

Як обрати кількість процесів

Питання: "Чи можна додавати процеси досхочу?"
Чи прискорить програму створення 100 процесів на 8-ядерному CPU?

НІ, ЦЕ ШКІДЛИВО

Обмеження CPU

БАЗОВА ФОРМУЛА	`N = cpu_count()`

Використання > 100% ядер викликає Context Switching

Витрати на перемикання
Якщо процесів більше ніж ядер, CPU витрачає час на їх перемикання, а не на роботу. Ефективність падає.

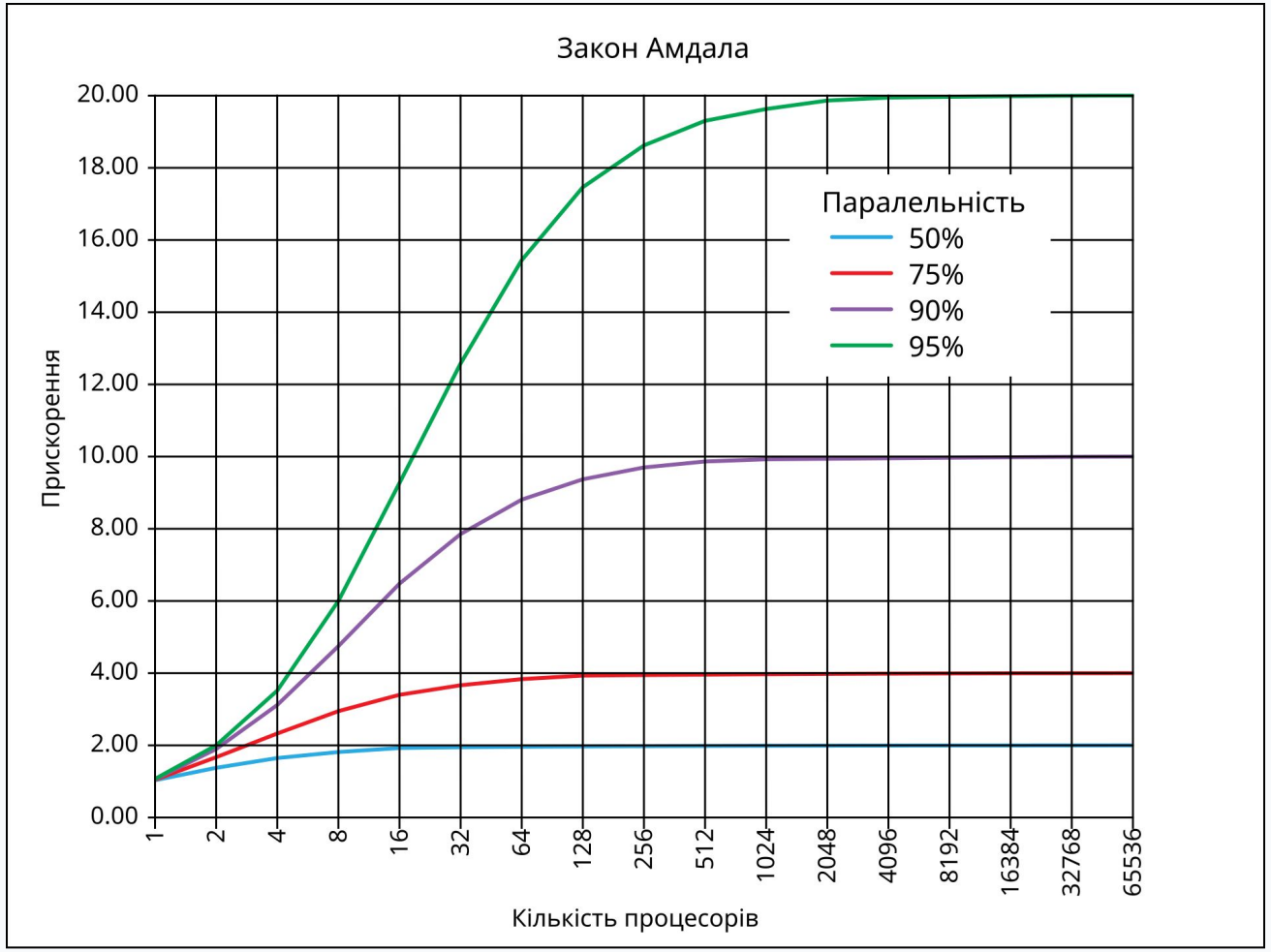
Пам'ять та Стабільність

БЕЗПЕЧНА ФОРМУЛА	`min(cpu_count(), X)`

Кожен процес - це копія інтерпретатора Python у пам'яті.
X - це допустимий ліміт пам'яті.

Рекомендація: Залиште 1 ядро
cpu_count() - 1 дозволяє ОС обробляти фонові задачі (мережа, миша, UI) без зависань.

Закон Амдала: Межі масштабування



ТЕОРЕТИЧНЕ ПРИСКОРЕННЯ

$$S = \frac{1}{P + \frac{1 - P}{N}}$$

S — **Speedup (Прискорення)**
У скільки разів швидше виконується задача.

P — **Serial Portion (Послідовна частина)**
Частка коду, яку неможливо розпаралелити (напр. IPC, ініціалізація).

N — **Processors (Процеси)**
Кількість паралельних виконань.

Реальність проти теорії

Якщо лише 5% коду є послідовним (P=0.05), максимальне прискорення ніколи не перевищить 20x, навіть якщо ви запустите 1000 процесів. IPC збільшує P!

Міжпроцесна взаємодія (IPC)

Queue
Безпечна FIFO черга для передачі об'єктів.

Speed: MEDIUM
Use: **Producer-Consumer**

Pipe
Швидкий дуплексний канал між двома процесами.

Speed: HIGH
Use: **1-to-1 Fast Data**

Shared Memory
Спільний блок пам'яті без копіювання.

Speed: ULTRA
Use: **Large Arrays/NumPy**

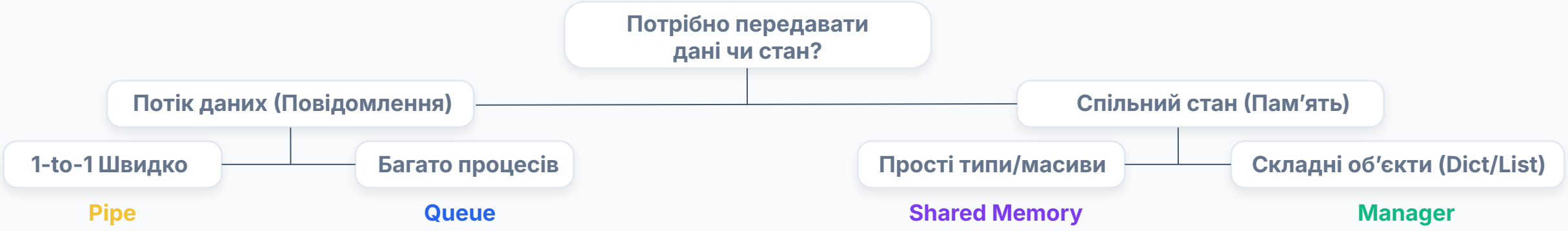
Manager
Спільні складні об'єкти (dict, list) через проксі.

Speed: LOW
Use: **Shared State/Dicts**

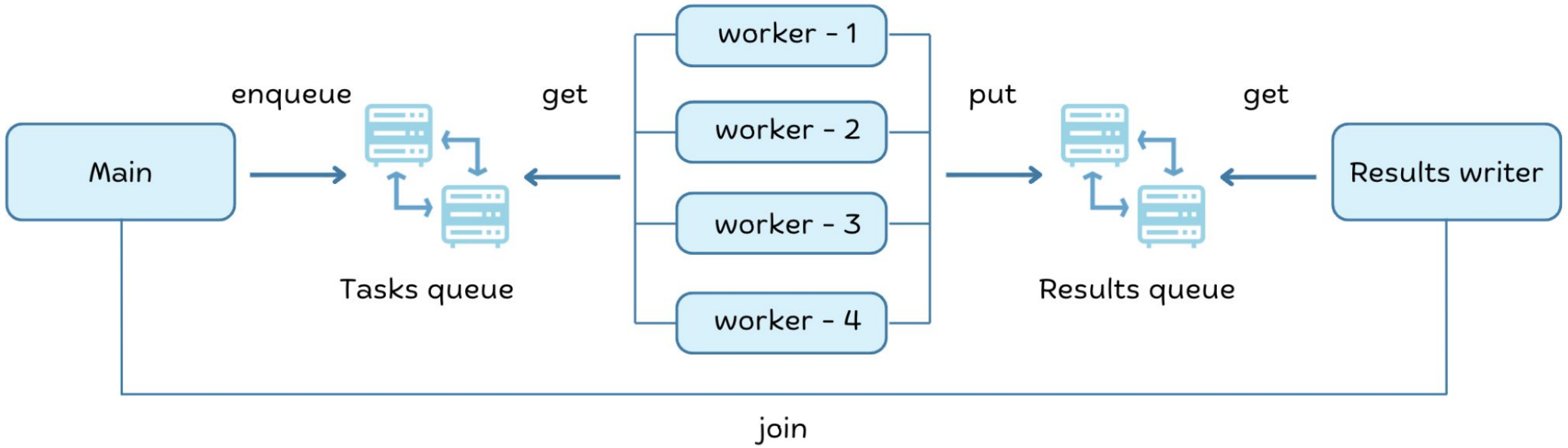
Sockets
Мережева взаємодія, навіть між різними машинами.

Speed: VARIES
Use: **Distributed Systems**

Як обрати метод?



IPC: Черги (Queue)



multiprocessing.Queue

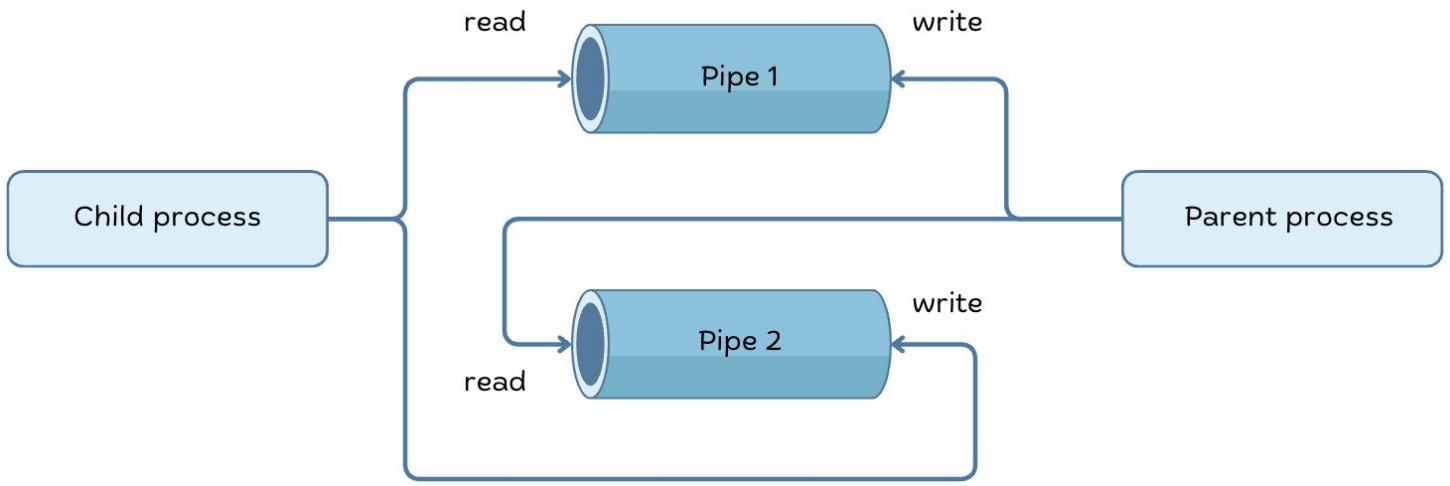
Потоко- та процесо-безпечна черга FIFO. Дозволяє передавати будь-які Python об'єкти між процесами через механізм серіалізації.

Backpressure (Maxsize)

Якщо задано maxsize, метод put() заблокує Producer, поки в черзі не з'явиться місце.

```
from multiprocessing import Process, Queue

def worker(q):
   data = q.get() # Unpickles object
   print(f"Got: {data}")

if __name__ == "__main__":
   q = Queue(maxsize=5)
   p = Process(target=worker, args=(q,))
   p.start()
   q.put([1, 2, 3]) # Pickles list
```

IPC: Канали (Pipe)



Прямий канал зв'язку

Duplex (Двосторонній)
Дані можуть передаватися в обох напрямках одночасно.

Швидкий для пар
Швидше за Queue, бо працює лише для двох процесів.

Обережно з порядком
Небезпечно, якщо >1 процесу читають/пишуть в один кінець одночасно.

Двосторонній зв'язок
Метод Pipe() повертає два об'єкти з'єднання. Кожен кінець може і відправляти, і приймати дані (Full-Duplex).

Серіалізація (Pickle)
Передача об'єктів вимагає їх перетворення в байти. Це створює навантаження на CPU при великих обсягах даних.

Блокування та Буфер
Якщо буфер ОС заповнений, send() заблокує процес. recv() блокує процес, поки дані не надійдуть.

"Pipe працює як телефонний дріт: ви говорите в один кінець, а слухаєте з іншого. Але пам'ятайте про обмежену пропускну здатність лінії."

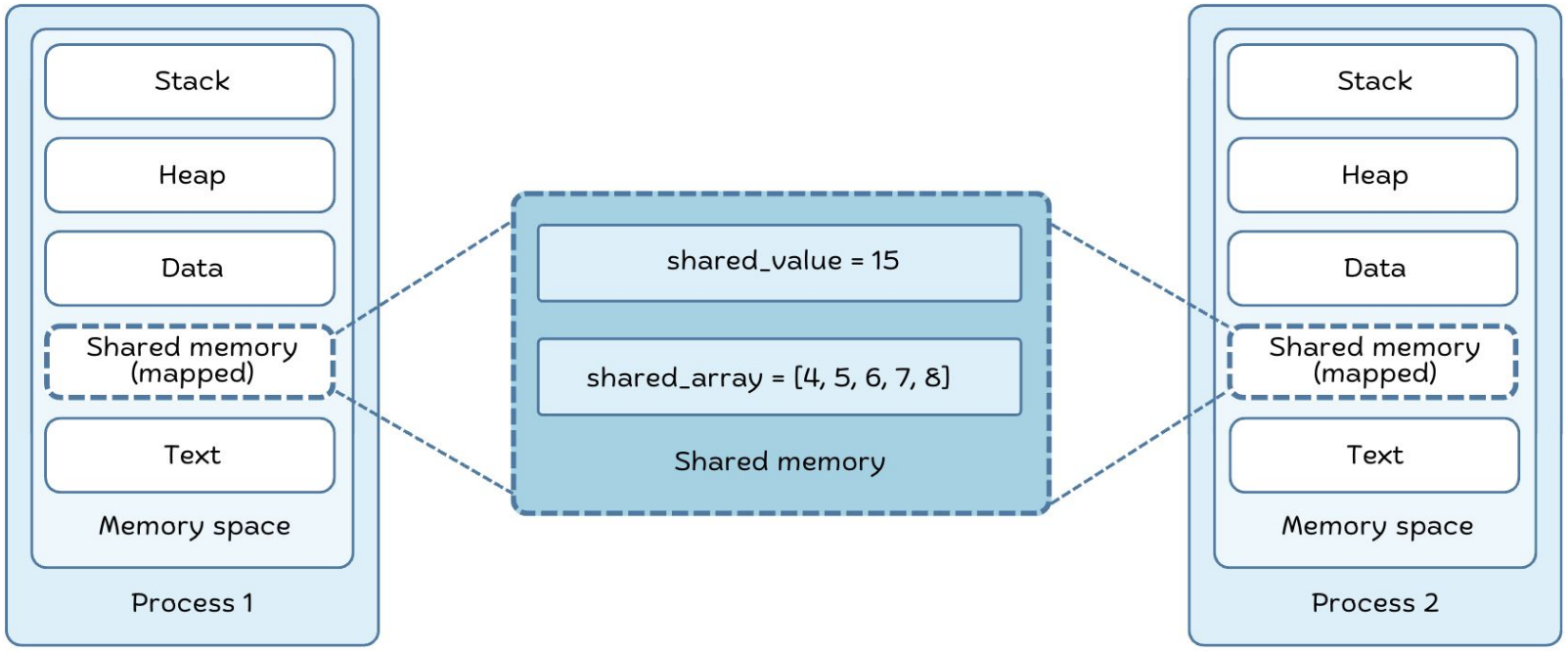
КОЛИ ВИКОРИСТОВУВАТИ?

- 1-ТО-1 ОБМІН
- ШВИДКІ КОМАНДИ
- МІНІМУМ OVERHEAD

pipe_example.py

```
from multiprocessing import Process, Pipe

def worker(conn):
  conn.send([42, None, 'hello'])
  conn.close()

if __name__ == '__main__':
  parent_conn, child_conn = Pipe()
  p = Process(target=worker, args=(child_conn,))
  p.start()
  print(parent_conn.recv()) # -> [42, None, 'hello']
```

IPC: Спільна пам'ять (Shared Memory)



Найшвидший IPC

Дозволяє процесам працювати з одним блоком пам'яті напряму, уникаючи повільної серіалізації (pickle) та копіювання даних.

multiprocessing.Value
Для простих типів (int, float).

multiprocessing.Array
Для масивів фіксованого розміру.

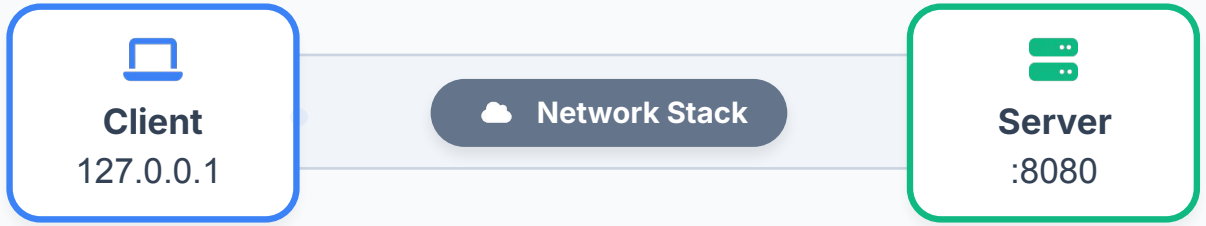
```
from multiprocessing import Value, Lock

# Спільна пам'ять (int)
counter = Value('i', 0)
lock = Lock()

with
lock:
  counter.value += 1
```

Ручна синхронізація

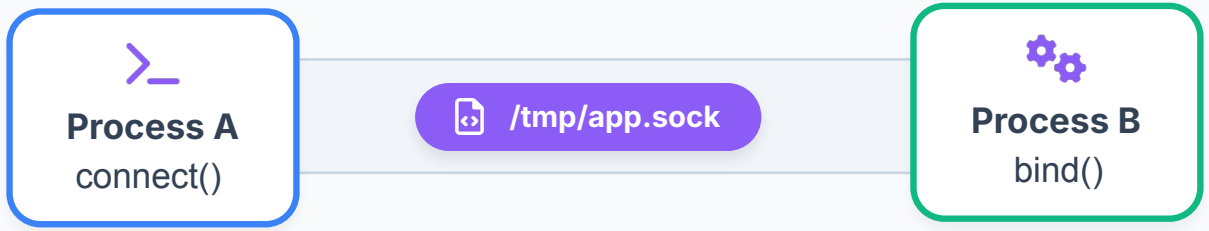
Операції не є атомарними. Обов'язково використовуйте Lock, інакше отримаєте Race Condition.

IPC: Сокети (Sockets)

1. TCP/IP (NETWORK)



2. UNIX DOMAIN SOCKETS



INET (TCP/UDP)
Універсально. Працює через мережу. Вищий overhead (заголовки, checksums).

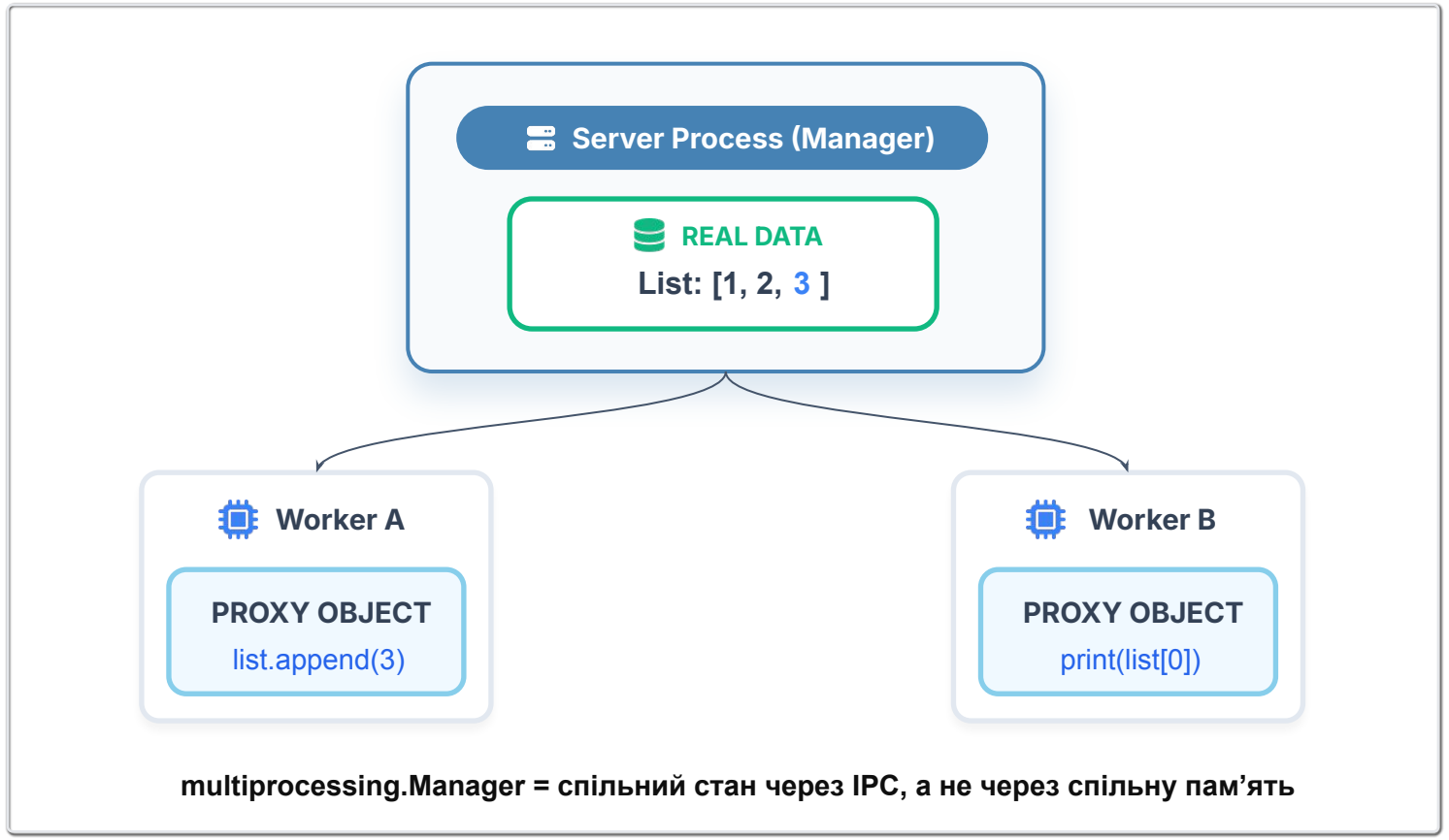
UNIX Domain
Тільки локально (Linux/macOS). Швидше: копіювання через буфер ядра.

Python socket example

```python
# 1. Мережевий сокет (Network)
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(('localhost', 8080))

# 2. Локальний сокет (Unix Domain) - Faster!
if hasattr(socket, 'AF_UNIX'):
   s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
   s.bind('/tmp/process.sock')
```

Важливо: AF_UNIX не працює на Windows (до останніх версій Win10/11), тому для кросплатформності краще використовувати localhost TCP.

Manager: Високорівневий обмін



Архітектура Manager

Менеджер запускає окремий серверний процес, який тримає реальні об'єкти. Інші процеси взаємодіють з ними через проксі-посилання, використовуючи мережеві сокети або канали.

- ✓ Гнучкість
- ✓ dict/list
- Повільніше

Використання: Ідеально підходить, коли потрібно розділити складні об'єкти (вкладені словники, списки), які не підтримуються `Value` або `Array`.

manager_example.py

```
from multiprocessing import Manager

with Manager() as manager:
   # Створення спільних структур
   shared_list = manager.list()
   shared_dict = manager.dict()

   p = Process(target=worker,
            args=(shared_list,))
   p.start()
   p.join()
```

Порівняння IPC методів

МЕТОД / ТИП	ЗРУЧНІСТЬ (API)	ШВИДКІСТЬ (SPEED)	БЕЗПЕКА (SAFETY)
Queue `Serialized`	★★★★★ Максимальна простота	Medium (Pickling overhead)	Thread-Safe
Pipe `Duplex`	★★★☆☆ Середня (Connection)	High (Direct link)	1-to-1 Only
Shared Memory `Raw Data`	★★☆☆☆ Низька (Manual)	Very High (Zero-copy)	Unsafe (Race)
Manager `Proxy`	★★★★☆ Висока (Dict/List)	Low (Server overhead)	Managed

Рекомендація

Queue - найкращий вибір за замовчуванням: баланс між зручністю та безпекою. Shared Memory - використовуйте лише для передачі великих масивів (NumPy/статистика).

BEST PRACTICE

Start with Queue

Практичні кейси

> Процеси - це єдиний надійний спосіб завантажити всі ядра CPU для виконання Python-коду, обходячи обмеження GIL.

Обробка Медіа

Ресайз зображень, конвертація відео, рендеринг PDF. Кожен кадр або сторінка обробляється окремим процесом.

Наукові обчислення

Криптографія (RSA, хешування), симуляції Монте-Карло, Machine Learning інференс. Задачі, де 99% часу — це робота CPU.

Map-Reduce Патерни

Розбиття гігантських датасетів на частини (chunks), паралельна обробка та фінальне об'єднання (aggregation) результатів.

Ізоляція помилок

Запуск ненадійного коду (плагінів, C-розширень) в окремому процесі. Якщо він впаде (SegFault), основна програма виживе.

ПАРАЛЕЛЬНА ОБРОБКА

Прискорення: Час виконання зменшується майже лінійно з додаванням ядер ($T \approx T_{total} / N_{cores}$), за винятком накладних витрат на IPC.

Ключові тези та рекомендації

01

Процеси для CPU-Bound

Використовуйте multiprocessing для важких обчислень, ML та криптографії. Це єдиний надійний спосіб обійти GIL і завантажити всі ядра процесора, попри накладні витрати на старт.

02

Стратегія вибору IPC

Починайте з Queue для безпеки. Якщо потрібна швидкість — використовуйте Pipe. Для масивних даних (numpy) — SharedMemory, але пам'ятайте про складність синхронізації.

03

Закон Амдала

Пам'ятайте про межі масштабування. Прискорення обмежується послідовною частиною коду. Оптимізуйте алгоритм перед додаванням нових ядер.

04

Спочатку профілювання

Витрати на серіалізацію (pickle) можуть "з'їсти" весь виграш. Використовуйте viztracer для пошуку вузьких місць в IPC.

"Потоки - для очікування (I/O), Процеси - для роботи (CPU). Розуміння цієї відмінності - ключ до продуктивного Python коду."

FINAL THOUGHT

Що почитати?

- Multiprocessing
- Multiprocessing - Manage Processes Like Threads
- Speed Up Your Python Program With Concurrency
- Passing Messages to Processes
- ProcessPoolExecutor in Python: The Complete Guide
- Linux manual page - fork